# HUST

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.




ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

# THUẬT TOÁN DI TRUYỀN GIẢI BÀI TOÁN NGƯỜI BÁN HÀNG DU LỊCH

**Người thực hiện:**

Lê Anh Quân

**Đại học Bách khoa Hà Nội**

ONE LOVE. ONE FUTURE.

# Nội dung trình bày I

1. Giới thiệu bài toán TSP
2. Thuật toán Genetic Algorithm
3. Cài đặt thuật toán
4. Kết quả benchmark

# 1. Bài toán người bán hàng du lịch

**Phát biểu bài toán (TSP – Traveling Salesman Problem):**

- Cho $n$ thành phố và ma trận khoảng cách $D = (d_{ij})_{n \times n}$.
- Tìm **chu trình (tour) Hamilton** đi qua mỗi thành phố đúng một lần rồi quay về thành phố xuất phát, sao cho **tổng chi phí nhỏ nhất**.

**Input / Output / Ràng buộc:**

- **Input:** ma trận khoảng cách (hoặc toạ độ thành phố), thỏa $d_{ij} \geq 0$.
- **Output:** hoán vị $s = (s_1, s_2, \ldots, s_n)$ biểu diễn một tour khép kín.
- **Hàm mục tiêu:**

$$f(s) = \sum_{i=1}^{n-1} d_{s_i, s_{i+1}} + d_{s_n, s_1} \tag{1}$$

- **Ràng buộc:** mỗi đỉnh được thăm đúng một lần trừ đỉnh bắt đầu, chu trình là vòng lặp đóng.

# 1. Độ phức tạp và các biến thể

**Vì sao TSP khó?**

- Không gian tìm kiếm: $(n!)$ tour
- TSP thuộc lớp NP-khó: chưa có thuật toán đa thức chính xác cho trường hợp tổng quát.
- $n = 25$ đã có $\approx 3 \times 10^{23}$ tour – duyệt toàn bộ là bất khả thi.

**Các biến thể thường gặp:**

- **Symmetric TSP:** $d_{ij} = d_{ji}$.
- **Asymmetric TSP (ATSP):** $d_{ij} \neq d_{ji}$ (miêu tả giao thông một chiều).
- **Metric TSP:** $d_{ik} \leq d_{ij} + d_{jk}$ (thường các bài toán với toạ độ thực thoả mãn).
- **Multi / Generalized TSP:** nhiều người bán hàng (mTSP), đi qua nhóm đỉnh (GTSP).
- **TSP có ràng buộc:** thời gian giao hàng (TSPTW), sức chứa xe (VRP).

# 1. Ứng dụng của TSP trong tối ưu mạng

**TSP xuất hiện khắp nơi trong network optimization:**

- **Tuyến cáp / drone:** lập lộ trình cho drone kiểm tra đường dây, ăng-ten
- **Gathering data in networks:** sensor network thu thập dữ liệu từ các nút khi các nút này kết nối qua các điểm thay đổi liên tục.
- **Định tuyến mạng:** dữ liệu cần đi qua một chuỗi các nút trung gian – tìm chu trình ngắn nhất thăm tất cả các nút.

**Ý nghĩa thực tiễn:**

- Trong mạng, chi phí có thể là **độ trễ**, **băng thông tiêu thụ** hoặc **năng lượng**.
- Giải tốt TSP $\Rightarrow$ giảm độ trễ tổng, giảm năng lượng, tăng tuổi thọ mạng.
- Là bài toán con trung tâm của **VRP (Vehicle Routing Problem)** – điều phối đội xe, logistics.

# 2. Ý tưởng thuật toán di truyền

**Truyền cảm hứng từ tiến hóa tự nhiên ( Darwin ):**

- Cá thể phù hợp môi trường hơn có xác suất sống sót và sinh sản cao hơn.
- Thế hệ sau kết hợp vật chất di truyền của cha mẹ + đột biến $\Rightarrow$ dần dần tiến hóa.

**Áp vào bài toán tối ưu:**

- Mỗi **cá thể (individual)** = một lời giải (một tour TSP).
- **Quần thể (population)** = tập nhiều lời giải song song.
- **Độ thích nghi (fitness)** = chất lượng lời giải (chi phí tour càng nhỏ càng tốt).
- Lặp lại: **chọn lọc** $\rightarrow$ **lai ghép** $\rightarrow$ **đột biến** qua nhiều thế hệ.

Ưu điểm với TSP: không cần gradient/đạo hàm, dễ song song hoá, tìm kiếm toàn cục trên không gian hoán vị khổng lồ.

# 2. Giải thuật di truyền: Mô hình cơ bản nhất

**Ý tưởng cốt lõi:** Cải thiện giải pháp dựa trên mô phỏng tiến hóa tự nhiên.

- **Mã hoá (Encoding):** Mỗi cá thể là một hoán vị $\pi$ của các thành phố (một đường đi).
- **Đánh giá (Fitness):** Chi phí đường đi càng nhỏ $\rightarrow$ độ thích nghi càng cao.

**Vòng lặp tiến hoá đơn giản:**

1. **Khởi tạo:** Sinh ngẫu nhiên một quần thể gồm các đường đi.
2. **Chọn lọc:** Chọn các cá thể có fitness tốt để làm cha mẹ.
3. **Lai ghép:** Kết hợp các đoạn đường đi của cha mẹ để sinh ra con.
4. **Đột biến:** Thay đổi ngẫu nhiên một vài đỉnh trong đường đi của con.
5. **Lặp lại** từ bước 2 đến khi đạt số thế hệ tối đa.

# 2. Các yếu tố cải tiến cho bài toán TSP

**Vấn đề:** GA cơ bản dễ làm mất nghiệm tốt hoặc tạo ra đường đi không hợp lệ (trùng lặp đỉnh khi lai ghép).

**Các cơ chế nâng cao:**

- **Chọn lọc Tournament:** Chọn ngẫu nhiên $k$ cá thể, lấy con tốt nhất làm cha mẹ $\rightarrow$ kiểm soát áp lực tiến hoá.
- **Lai ghép chuyên dụng (Crossover):** Giữ cho con sinh ra luôn là hoán vị hợp lệ.
  - **OX (Ordered):** Lấy một đoạn từ cha 1, điền phần còn lại theo thứ tự xuất hiện ở cha 2.
  - **PMX (Partially Mapped):** Tráo đoạn gen, dùng ánh xạ để xử lý các đỉnh trùng lặp.
- **Đột biến Swap (Xác suất $p_m$):** Hoán đổi ngẫu nhiên 2 vị trí $\rightarrow$ duy trì sự đa dạng sinh học.
- **Chủ nghĩa tinh hoa (Elitism):** Giữ lại trực tiếp $E$ cá thể xuất sắc nhất sang thế hệ sau $\rightarrow$ đảm bảo không thoái hoá.

# 2. Minh hoạ Lai ghép: Ordered Crossover (OX)

**Ví dụ với $n = 8$ thành phố. Chọn 2 điểm cắt ngẫu nhiên:**

**Cha 1:** 1 2 | 3 4 5 | 6 7 8 $\rightarrow$ Đoạn giữa: 3, 4, 5

**Cha 2:** 2 4 | 6 8 7 | 1 3 5

**Các bước thực hiện OX:**

- **Bước 1 (Chép):** Giữ nguyên đoạn giữa của Cha 1.
  $\rightarrow$ Con: _ _ | 3 4 5 | _ _ _
- **Bước 2 (Lọc):** Lấy các gen của Cha 2 theo thứ tự, loại bỏ các gen đã chép (3, 4, 5).
  $\rightarrow$ Dãy còn lại: 2, 6, 8, 7, 1
- **Bước 3 (Điền):** Điền dãy trên vào khoảng trống của Con, bắt đầu từ sau điểm cắt 2, vòng lại từ đầu.
  $\rightarrow$ **Con OX:** 7 1 | 3 4 5 | 2 6 8

# 2. Minh hoạ Lai ghép: Partially Mapped (PMX)

**Cha 1:** 1 2 | 3 4 5 | 6 7 8 **Cha 2:** 2 4 | 6 8 7 | 1 3 5

**Các bước thực hiện PMX:**

- **Bước 1 (Chép):** Giữ nguyên đoạn giữa của Cha 1.
  → Con: _ _ | 3 4 5 | _ _ _
- **Bước 2 (Ánh xạ):** Khớp các gen ở đoạn cắt để tạo ánh xạ thay thế:
  → $3 \leftrightarrow 6, 4 \leftrightarrow 8, 5 \leftrightarrow 7$
- **Bước 3 (Điền):** Hạ các gen ngoài đoạn cắt của Cha 2 xuống Con. Nếu gen bị trùng, dùng ánh xạ để đổi tiếp đến khi không trùng:
  - Hai vị trí đầu (2, 4): 2 giữ nguyên, 4 ánh xạ thành 8.
  - Ba vị trí cuối (1, 3, 5): 1 giữ nguyên, 3 thành 6, 5 thành 7.

  → **Kết quả (Con PMX):** 2 8 | 3 4 5 | 1 6 7

# 2. Mô hình giải thuật đầy đủ

**Vòng lặp tiến hoá hoàn chỉnh giải TSP:**

1. **Khởi tạo:** Quần thể $P$ gồm 1 hoán vị cơ sở và $(P-1)$ ngẫu nhiên.
2. **Đánh giá & Sắp xếp:** Giữ lại $E$ cá thể tốt nhất sang thế hệ mới (*Elitism*).
3. **Sinh sản** để lấp đầy $(P-E)$ cá thể còn lại:
   - Chọn 2 cha mẹ bằng *Tournament* (kích thước $k$).
   - Lai ghép *OX/PMX* ($p_c$) và Đột biến con sinh ra ($p_m$).
   - Thêm con vào quần thể thế hệ mới.
4. **Chuyển tiếp:** Lặp lại bước 2 đến khi đạt $G$ thế hệ.
5. **Kết thúc:** Trả về đường đi tốt nhất tìm được.

**Siêu tham số:** $P$ (quần thể), $G$ (thế hệ), $p_m, p_c$ (đột biến, lai ghép), $E$ (elitism), $k$ (tournament), loại crossover.

# 3. Cài đặt cụ thể – kiến trúc chương trình

**Khung C++17, mô hình hoá hướng đối tượng:**

- `TSPProblem`: đọc file TSPLIB (MatrixLoader), lưu ma trận khoảng cách, hàm `tourCost()`.
- `TSPSolver` (interface) $\rightarrow$ `GreedySolver` và `GeneticAlgorithm`.
- `GeneticAlgorithmConfig`: $P$, $G$, $p_m$, $p_c$, elitism, tournament, seed, OX/PMX.
- `SolverResult`: tour, chi phí, thời gian, gap% so với tối ưu đã biết.

**Chi tiết mã nguồn:**

- RNG: `std::mt19937_64`, seed dẫn xuất từ (config, instance, run) $\Rightarrow$ kết quả tái lập được.
- Độ phức tạp mỗi thế hệ: $O(P \cdot n \log n)$ (sort) $+O(P \cdot n)$ (OX/đánh giá).

# 3. Giả mã thuật toán

**Algorithm 1** GeneticAlgorithm::solve(problem, config)

$P \leftarrow \{\text{identity}\} \cup \{P_{\text{size}} - 1 \text{ hoán vị ngẫu nhiên}\}$
**for** $g = 1 \ldots G$ **do**
    Sort($P$) theo chi phí tăng dần
    $P' \leftarrow E$ cá thể tốt nhất ▷ elitism
    **while** $|P'| < P_{\text{size}}$ **do**
        $p_1, p_2 \leftarrow \text{TournamentSelect}(P, k)$
        $c \leftarrow$ **with prob.** $p_c$: Crossover$(p_1, p_2)$ **else** copy$(p_1)$
        **with prob.** $p_m$: swap 2 gen của $c$ ▷ mutation
        $P' \leftarrow P' \cup \{c\}$
    **end while**
    $P \leftarrow P'$
**end for**
**return** cá thể tốt nhất của $P$

# 4. Thiết kế benchmark

**Dữ liệu TSPLIB, chia theo nhóm kích thước:**

- Lớp `0–100`: 22 instance (14–99 thành phố); Lớp `100–200`: 25 instance.
- Tổng 112 instance TSPLIB từ 14 đến 85.900 thành phố.

**Phương pháp:**

- Baseline: **Greedy (Nearest Neighbour, start = 0)** – deterministic, chạy 1 lần/instance.
- GA: ngẫu nhiên $\Rightarrow$ chạy 5 seed mỗi (config, instance), lấy mean/best/std của gap%.
- Gap% $= \frac{\text{cost}-\text{opt}}{\text{opt}} \times 100\%$.
- Mỗi config định nghĩa trong CSV: `label`, `population`, `generations`, $p_m$, $p_c$, `E`, `k`, `OX/PMX`.

# 4. Kết quả lớp 0–100:

## Config ranking theo mean gap% (5 seeds)

| Config | Mean gap% | Best gap% | Thời gian |
|---|---|---|---|
| **big_200x900_m40_x99_e5** | **6.71** | 0.00 | 74 ms |
| big_200x900_m30_x99_e5 | 7.94 | 0.00 | 73 ms |
| r1_mut_0.40 | 9.38 | 0.00 | 36 ms |
| c_m40_x99_e5 | 9.40 | 0.00 | 73 ms |
| baseline | 14.25 | 0.00 | 34 ms |
| Greedy(start=0) | 24.83 | 4.89 | 0.01 ms |

## Kết quả chính

- GA thắng **22/22** instance $(100\%)$ so với Greedy.
- Mean gap: **6.71%** vs **24.83%** $\Rightarrow$ tốt hơn $\approx 3.7$ lần.
- 8/22 instance GA tìm đúng tối ưu (gap = 0%).

# 4. Quá trình tinh chỉnh tham số (0–100)

**Hai vòng thử, mỗi vòng giữ ngân sách bằng nhau để so sánh công bằng:**

- **Vòng 1 – thay tham số:** $p_m \in \{0.1, 0.2, 0.4\}$, $p_c \in \{0.9, 0.99\}$, elitism $\in \{2, 5, 10\} \Rightarrow$ best: $p_m = 0.4$, $p_c = 0.99$, $E = 5$.
- **Vòng 2 – tăng budget:** $200 \times 900$, $200 \times 1200$, $300 \times 600 \Rightarrow$ best: `big_200x900_m40_x99_e5`.

**Bài học từ kết quả:**

- Mutation cao (0.4) lại thắng – vì implementation chỉ đột biến **tối đa 1 lần/con**, giá trị cao vẫn an toàn và duy trì đa dạng.
- Crossover rate 0.99: hầu hết con lai đều sinh ra từ OX.
- Elitism 5: vừa bảo vệ best, vừa không thu hẹp đa dạng quá mạnh.

# 4. Kết quả lớp 100–200 và giới hạn

**Hai vòng sweep trên 25 instance (100–198 thành phố):**

- Vòng 1 (budget nhỏ): GA chỉ thắng 4/25; sang Vòng 2 (budget lớn hơn): 5/25.
- Vòng 2 – tăng budget lên $150 \times 4000$ và $400 \times 3000$: GA thắng **14/25 (56%)**, mean gap **21.61%** (best config R_t3) so với Greedy 43.91%.

**Phân tích - vì sao khó hơn:**

- Không gian hoán số tăng bật thang $\Rightarrow$ cùng budget cho chất lượng thấp hơn rõ rệt.
- Greedy có thể khôgn tối ưu: `brg180` (greedy 533%!) – GA luôn thắng lớn ở đây.
- Ngược lại, trên instance có cấu trúc gần tối ưu sẵn (`pr107`, `pr144`, `si175`, `bier127`), Greedy rất khó bị đánh bại.
- GA thuần (không local search 2-opt) khó đạt gap nhỏ khi $n \geq 1000$; kết quả khuyến nghị kết hợp **memetic (GA + 2-opt)**.

# 4. Kết luận

- **GA > Greedy rõ ràng** ở lớp nhỏ: thắng 22/22, mean gap giảm từ 24.83% xuống 6.71%.
- **Chất lượng tỷ lệ nghịch với độ lớn**: $n \geq 100$ cần budget lớn hơn nhiều lần để duy trì ưu thế.
- **Tham số quan trọng nhất** (theo benchmark): mutation rate, crossover rate, elitism, và budget (population $\times$ generations).
- **Hướng phát triển:** hybrid GA + 2-opt (memetic), greedy initialization, parallel evaluation.






# THÔNG TIN LIÊN HỆ

**Nhóm nghiên cứu:** TSP Solver & Benchmark Framework
**Trường:** Đại học Bách khoa Hà Nội

- **Github:** `https://github.com/` – TSP Solver & Benchmark (C++17)
- **Dữ liệu:** TSPLIB – `http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/`

*Xin chân thành cảm ơn Hội đồng đã lắng nghe!*

# Tài liệu tham khảo I


[1] G. Reinelt, *TSPLIB – a Traveling Salesman Problem Library*, INFORMS Journal on Computing, vol. 3, no. 4, pp. 376–384, 1991.

[2] J. H. Holland, *Adaptation in Natural and Artificial Systems*, University of Michigan Press, 1975.

[3] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, 1989.

[4] I. M. Oliver, D. J. Smith, and J. R. C. Holland, "A study of permutation crossover operators on the traveling salesman problem," in *Proc. International Conference on Genetic Algorithms*, pp. 224–230, 1987.

[5] D. E. Goldberg and R. Lingle, "Alleles, loci, and the traveling salesman problem," in *Proc. International Conference on Genetic Algorithms*, pp. 154–159, 1985.

# Tài liệu tham khảo II


[6] R. M. Karp, “Reducibility among combinatorial problems,” in *Complexity of Computer Computations*, Plenum Press, pp. 85–103, 1972.

[7] P. Toth and D. Vigo (eds.), *Vehicle Routing: Problems, Methods, and Applications*, SIAM, 2014.




# THANK YOU !